ARCHITECTURE REFERENCE

Sistema de Transportes de Moçambique

Software architecture of the passenger, driver and administration platform for urban public transport in Maputo and Matola: applications, technology stack, data model, runtime flows, deployment, security and technical debt.



SOURCE SNAPSHOT
main @ 57b723f (26 May 2026)

DOCUMENT DATE
15 October 2026

STATUS UPDATE
branch tm-assess, 15 October 2026 (see § 1.2)

SCOPE
transport-client · root app · transport-admin
transport-driver · PostgreSQL · USSD/SMS

Contents

1 Summary . . . 3
1.1 Architecture at a glance . . . 3
1.2 Status after the October 2026 assessment . . . 3
2 System Context . . . 5
2.1 Actors . . . 5
2.2 External systems . . . 6
3 Repository Layout . . . 7
4 Technology Stack . . . 8
4.1 Next.js 16 (App Router) . . . 8
4.2 React 19 and TypeScript . . . 9
4.3 Tailwind CSS v4 . . . 9
4.4 Prisma ORM and PostgreSQL . . . 9
4.5 Maps: MapLibre GL and OpenFreeMap . . . 9
4.6 three.js . . . 9
4.7 OSRM routing . . . 9
4.8 USSD and SMS . . . 10
4.9 Authentication building blocks . . . 10
4.10 Testing and tooling . . . 10
5 Application Architecture . . . 11
5.1 transport-client — passenger app (production) . . . 12
5.2 Root app — parallel passenger app and backend hub . . . 12
5.3 transport-admin — back-office . . . 12
5.4 transport-driver — driver portal . . . 12
6 Data Architecture . . . 13
6.1 Entity catalogue . . . 14
6.2 Encoding conventions . . . 14
6.3 Schema drift . . . 14
7 Runtime Behaviour . . . 15
7.1 Bus position simulation . . . 15
7.2 Passenger search and ETA . . . 16
7.3 Tracking page . . . 16
7.4 USSD session engine . . . 17
7.5 Driver portal login . . . 18
7.6 Administration . . . 18
8 Domain Calculations . . . 19
9 Deployment and Operations . . . 20
9.1 Environments . . . 20
9.2 Configuration . . . 21
9.3 Local ports . . . 21
9.4 Observability . . . 21
10 Security Assessment . . . 22
11 Quality Attributes and Technical Debt . . . 23
12 Recommended Roadmap . . . 24
13 Key Findings and Future Improvements . . . 25
13.1 Key findings . . . 25
13.2 Future improvements . . . 25
A API Catalogue . . . 26
A.1 transport-client . . . 26
A.2 Root app . . . 26
A.3 transport-admin . . . 27
B Glossary (Portuguese → English) . . . 28

1 Summary

Sistema de Transportes de Moçambique is a public-transport information platform for the Maputo–Matola area. Passengers find the buses serving a stop, see them on a map, get an arrival estimate and a fare, save stops for SMS alerts and do the same from any basic phone over USSD. Administrators maintain the network and fleet, and drivers get a lightweight portal.

4
Next.js 16 applications in one repository

73
API route handlers (25 root · 18 client · 30 admin)

74
pages across the four apps

≈ 58 k
lines of TypeScript / TSX (excluding scripts)

14
Prisma models in one PostgreSQL database

3
channels: web, USSD, SMS

1.1 Architecture at a glance

- **Style.** Four independent Next.js 16 App Router applications, each a small full-stack monolith (React UI plus Route Handlers as its backend). They share **one PostgreSQL database** through Prisma instead of calling each other; the only app-to-app HTTP calls are driver login (driver → root app) and admin reports (admin → root app).
- **Positions are simulated.** Bus locations are produced by an in-process simulator that animates each bus along an OSRM road geometry and writes them to the database every 30 s. No vehicle or driver device sends GPS data.
- **USSD is a first-class channel.** A stateless menu engine rebuilds the user's position in the menu from the `text` field sent by Africa's Talking on every hop, with pagination and SMS copies of results.
- **Production.** `transport-client` is deployed on Vercel (Root Directory = `transport-client`) and uses a Neon serverless PostgreSQL database in AWS us-east-1.

1.2 Status after the October 2026 assessment

The rest of this document describes the code at the source snapshot. An assessment that exercised every CRUD operation, the maps and every USSD menu path then changed the code on branch `tm-assess`. This table gives the current status of each finding; the commits hold the details.

Finding	§	Status	What changed
Admin APIs had no server-side authentication	10	**Fixed**	Login sets an HMAC-signed HttpOnly cookie; `proxy.ts` returns 401 on `/api/*` without it. Needs `ADMIN_SESSION_SECRET` in production (`a77b121`).
Public debug, bulk-write and seed endpoints	9.4, A	**Fixed**	Removed from client and admin; the root app keeps only the read-only seed status (`a77b121`, `c1519c1`).
Admin reports depended on the root app and its open `/api/admin/*`	5, 10	**Fixed**	Reports use the admin's own API and `/api/relatorios/stats` (`9aa0148`). The root app is not deployed.
Three fare formulas; stops ordered by id	6.1, 8	**Fixed**	One `Tarifa` rule and `estimateTrip` for web and USSD; stops ordered along the drawn path (`9599545`, `585ef35`, `3a2b9b5`).
Repeated arrival SMS	7.1	**Fixed**	`MISSION.notificadoEm`: at most one SMS per mission every 30 min (`8186dc6`).
Tracking page animated locally; client maps called OSRM	7.3	**Fixed**	Tracking re-reads `/api/bus/[id]` every 10 s and glides the marker; maps draw the stored path; the home map refreshes positions every 15 s (`585ef35`).
USSD screens over 182 characters, accents, fake payment	7.4, 10	**Fixed**	Screens cut to 182 GSM-7 characters; payment and PIN removed; real answers for every option (`64fc6f3`).
Driver passwords in USSD logs	10	**Fixed**	Masked in the request log (`086ce7f`).

Finding	§	Status	What changed
`availableOnly` filter on a non-existent column	4.2, 11	**Fixed**	Filters on the driver relation (`3a2b9b5`).
Africa's Talking SDK crashed `/api/ussd` without a key	—	**Fixed**	Client created on first use; `sendSMS` never throws (`8186dc6`).
Schema drift and migrations	4.4, 6.3	**Partly**	Root migrations now reproduce `prisma/schema.prisma` exactly (`8186dc6`); the client's `Proprietario` copy still differs (unused at runtime).
Driver portal wrote nothing back	5.4	**Partly**	Trip start / end shared with USSD through `/api/driver/trip` (`9599545`); GPS still not sent.
Master driver password `123456`, plain-text passwords	10	**Open**	Unchanged.
Passenger and driver-trip APIs trust ids from the request	10	**Open**	Unchanged; `/api/driver/trip` is new and has no authentication either.
Webhooks not authenticated; simulator in serverless memory	10, 11	**Open**	Unchanged.
Owner certificate upload is dropped by the admin API	—	**Open**	Needs file storage (e.g. Vercel Blob).

2 System Context

Who uses the system, through which channel, and which external services it depends on.

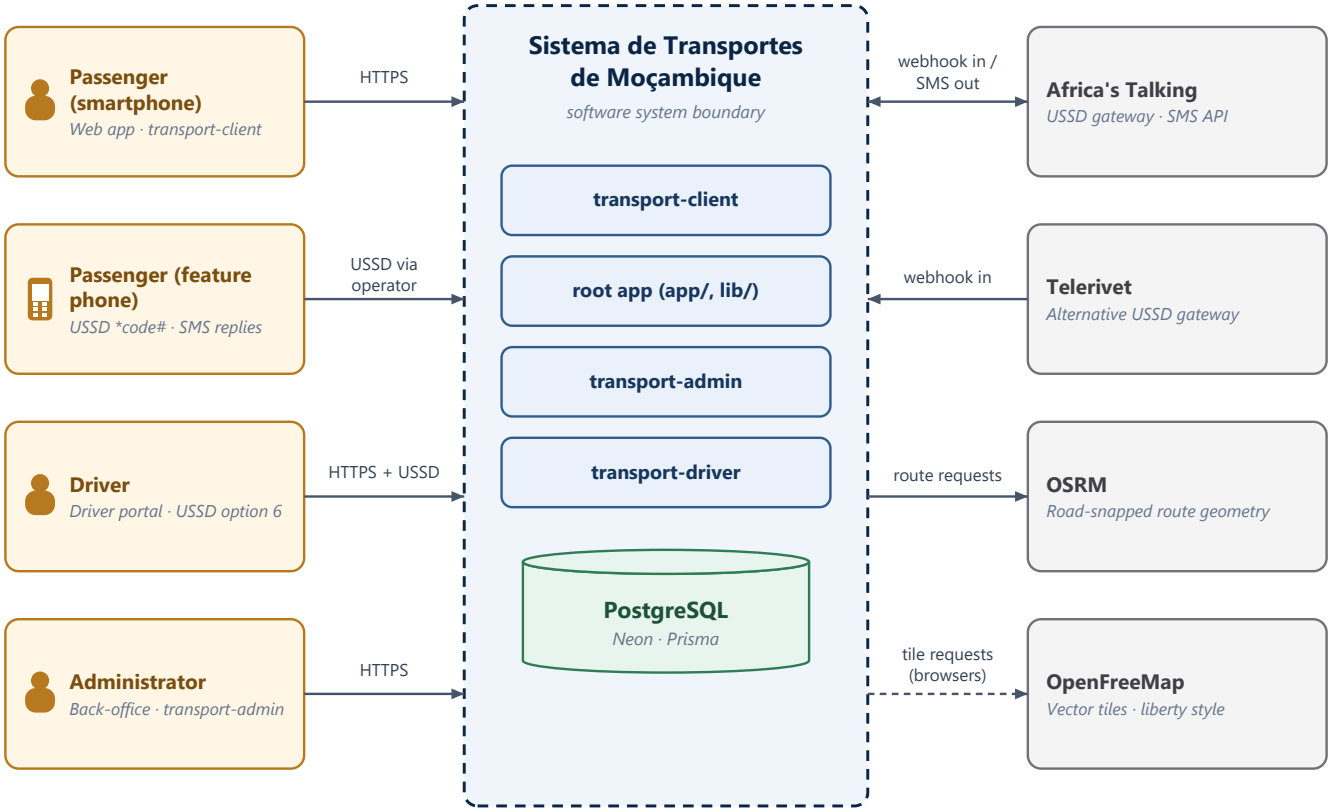


Figure 1 — System context. Solid arrows are requests; the dashed arrow is tile traffic that goes straight from browsers to OpenFreeMap.

2.1 Actors

Actor	Channel	What they can do
Passenger (smartphone)	`transport-client` web app over HTTPS	Register / log in with phone and password, choose município → via → paragem (→ destination), see buses on a 3D map with ETA and fare, track one bus, save stops ("missions") for SMS alerts, manage profile.
Passenger (feature phone)	USSD short code through Africa's Talking or Telerivet	Find a bus between two places, browse routes, list stops by area, compute a fare, track a bus by plate, read help; results are also sent by SMS.
Driver	`transport-driver` web app; USSD option 6	Log in with phone + password, see assigned bus, route and stops on a map, run a trip timer. Over USSD: start / end trip (sets `Motorista.status`), view own info, route and bus.
Administrator	`transport-admin` web app	Create and edit provinces, municipalities, routes, stops, buses, owners, drivers, passengers and admins; assign buses to routes, drivers and owners; dashboards and reports.

2.2 External systems

System	Role	Integration point
Africa's Talking	USSD gateway and SMS API	Webhook `POST /api/ussd` (form-encoded); SMS through the `africastalking` SDK in `lib/notifications.ts`
Telerivet	Alternative USSD gateway	Webhook `POST /api/ussd-telerivet`; JSON reply `{ messages, continue_session }`
OSRM (router.project-osrm.org)	Road-snapped route geometry	Server side in root `lib/busSimulator.ts`; browser side in the driver's route page
OpenFreeMap	Vector tiles and the liberty style	Loaded by MapLibre in every browser map
Neon	Managed serverless PostgreSQL	`DATABASE_URL`, the same value in the root, client and admin `.env`
Vercel	Hosting and serverless runtime	`transport-client` project; `vercel.json` at the repository root

3 Repository Layout

One Git repository, no workspace tooling: every application has its own package.json, node_modules, Prisma schema and lock file.

Path	Contents	Notes
`app/`, `lib/`, `prisma/`, `types/` (root)	Next.js app whose package.json is also named `transport-client`: passenger UI, legacy `/admin` UI and backend hub	25 route handlers, 14 pages; targeted by the root `vercel.json`
`transport-client/`	Passenger web app and USSD (deployed)	18 route handlers, 7 pages, latest USSD engine
`transport-admin/`	Administration back-office	30 route handlers, 47 pages, Jest tests, 4 own migrations
`transport-driver/`	Driver portal	6 pages, no backend, no Prisma
`docs/`	README pages (getting started, architecture, database) and this PDF	
`*.js` at root, `scripts/`, `transport-admin/scripts/`, `transport-client/*.js`	≈ 233 one-off maintenance scripts: seeding, importing Maputo / Matola stops, fixing associations, checks	Run with `node` / `tsx` directly against `DATABASE_URL`; not part of the runtime
`maputo-stops-data.json`, `matola-stops-data.json`, `neighborhood-mappings.*`	Reference data for stops and neighbourhoods	Used by import scripts
`docker-compose.yml`	Local PostgreSQL 16 on port 5433	Not referenced by the current `.env` files
`start-all.bat`, `restart-dev-server.bat`	Windows helpers that open one terminal per app	Port labels differ from the scripts (§ 9.3)
`scratch/`	Untracked USSD timing and debugging scripts	

Local clutter is mostly untracked

About 260 status markdown files in the working tree are ignored by `.gitignore` (`*.md` with an allow-list for README and `docs/`). They do not reach the repository, but they make the working directory hard to navigate.

4 Technology Stack

What each technology is, and exactly how this codebase uses it.

Layer	Technology	Version	Used by
Runtime	Node.js	≥ 20.9 required by Next 16 (docs still say 18)	all apps
Framework	Next.js App Router, Turbopack	16.2.4	all apps
UI	React, React DOM	19.2.4	all apps
Language	TypeScript (`strict`)	5.x	all apps
Styling	Tailwind CSS + `@tailwindcss/postcss`	4.x	all apps
ORM	Prisma Client + CLI	5.22	root, client, admin
Database	PostgreSQL	16 (Docker) / Neon	shared
Maps	MapLibre GL JS	5.24	client, root, driver, admin
Map data	OpenFreeMap (OpenMapTiles schema)	hosted	browsers
Routing	OSRM HTTP API	public demo server	root simulator, driver
3D	three.js	0.184	client, driver (unused components)
Charts	Recharts	3.8	admin
Telecom	`africastalking` SDK	0.7.3	root, client
Serverless	`@vercel/functions` (`waitUntil`)	3.5	client
Passwords	bcryptjs	3.0	root, client, admin
Tests	Jest + ts-jest	29	admin
Lint	ESLint flat config + eslint-config-next	9 / 16.2.4	all apps
Scripts	tsx, ts-node	4.21 / 10.9	seeds and maintenance
Infra	Docker Compose, `postgres:16-alpine`	—	local development

4.1 Next.js 16 (App Router)

- **Routing is the file system.** Every `app/**/page.tsx` is a page and every `app/api/**/route.ts` exports one function per HTTP verb (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`) that takes a Web `Request` / `NextRequest` and returns `NextResponse`. These Route Handlers are the whole backend; there is no separate API server.
- **Dynamic segments** follow the Next 15+ convention where `params` is a promise: `{ params }: { params: Promise<{ id: string }> }` followed by `await params`.
- **Rendering model.** Almost every page is a Client Component (`"use client"`) that fetches its own API in `useEffect`. Server Components, Server Actions and Next's data cache are not used, so each app behaves like a single-page app served by Next. Map components are loaded with `next/dynamic` and `ssr: false` because MapLibre needs `window`.
- **No `middleware.ts` / `proxy.ts`.** There is no request-level gate for authentication, CORS or rate limiting; each handler sets its own headers.
- **Build configuration.** The root `next.config.ts` sets `turbopack.root`, `experimental.optimizePackageImports` (`@prisma/client`, `maplibre-gl`), image formats, and also a custom `webpack()` hook and `swcMinify`. The webpack hook has no effect when Turbopack is the bundler (the default in Next 16) and `swcMinify` is no longer an option. The admin app raises the Node heap to 4 GB (`cross-env NODE_OPTIONS=--max-old-space-size=4096`) to build its 33 k-line codebase.

4.2 React 19 and TypeScript

- Identity lives in component state plus `localStorage` keys: `utente` (passenger), `motorista` (driver) and `admin`. The admin app wraps pages in `AuthContext` and `SidebarContext` providers and uses a `useDebounce` hook for search inputs.
- TypeScript runs in `strict` mode, but API code relies on `any` (`whereClause: any`, `driver as any`). That is how `transport-admin/app/api/transportes` can filter on a non-existent `motoristaId` column (it fails at runtime with `?availableOnly=true`).

4.3 Tailwind CSS v4

Configuration is CSS-first: `globals.css` starts with `@import "tailwindcss"` and is compiled by the `@tailwindcss/postcss` plugin. `transport-client` still carries a v3-style `tailwind.config.ts`, which v4 ignores unless it is referenced with `@config`.

4.4 Prisma ORM and PostgreSQL

- **Client lifecycle.** `lib/prisma.ts` keeps a single `PrismaClient` on `globalThis` in development so hot reloads do not open new pools, and logs only errors and warnings.
- **Anti-patterns.** Some handlers create their own `new PrismaClient()` (e.g. root `/api/driver/login`), and several call `prisma.$disconnect()` in `finally`. On the shared client this closes the pool under requests that are still running and forces a reconnect per request, which is expensive on Neon cold starts.
- **Keys.** Primary keys are `cuid()` strings; business codes (`codigo`, `matricula`, `bi`, `email`, `telefone`) carry unique constraints. Join tables use composite uniqueness and `onDelete: Cascade`.
- **Geodata as text.** Coordinates are stored as strings and parsed in JavaScript after loading rows. There is no PostGIS, no spatial index and no query-side distance filter; every proximity check loads candidate rows into memory.
- **Migrations.** The root, client and admin folders hold three independent migration histories (3, 2 and 4 migrations) for one database. The `_prisma_migrations` table can only match one of them, so `prisma migrate` from the other two will report drift.

4.5 Maps: MapLibre GL and OpenFreeMap

- MapLibre GL JS 5 renders vector tiles with WebGL. The base style is `https://tiles.openfreemap.org/styles/liberty`; a second source, `openmaptiles` (`tiles.openfreemap.org/planet`), feeds a `fill-extrusion` layer for 3D buildings driven by building height attributes.
- The camera is tilted (`pitch` 45° on the home map, 60° in `TransportMap` and the driver dashboard). Routes are GeoJSON `LineString` sources drawn as `line` layers in each `Via.cor`; buses and stops are HTML `maplibregl.Marker`s with `Popup`s, and `LngLatBounds` fits the view.
- Leftover dependencies: `leaflet` / `react-leaflet`, `mapbox-gl` and `@googlemaps/js-api-loader` are still in `package.json`, and a Google Maps key sits in `transport-client/.env.local`, but no code loads Google Maps.

4.6 three.js

`Bus3D.tsx` builds a procedural bus (box geometries, Phong materials, ambient and directional light) in its own `WebGLRenderer`, and `Bus3DMarker.tsx` wraps it as a map marker. No page imports either component, nor the driver's `SketchfabBus3D`, so three.js currently adds install weight only.

4.7 OSRM routing

Requests take the form `GET https://router.project-osrm.org/route/v1/driving/{lng,lat;…}?overview=full&geometries=geojson`. The server keeps results in a `Map` keyed by the waypoint list and falls back to straight segments when OSRM fails. The public demo server has no SLA and a fair-use policy; for production, self-host OSRM (or use a commercial router) and store the snapped geometry in `Via.geoLocationPath` once, instead of resolving it at runtime.

4.8 USSD and SMS

- **Africa's Talking USSD.** The gateway sends `application/x-www-form-urlencoded` fields `sessionId`, `serviceCode`, `phoneNumber` and `text`. `text` holds the whole session input joined by `*` (e.g. `1*7*3`). The reply is `text/plain` starting with `CON` (keep the session, show a menu) or `END` (close it).
- **Telerivet.** The gateway posts `event`, `id`, `content`, `from_number` and `secret`; the handler reuses the same menu logic and answers with JSON.
- **SMS.** `africastalking.SMS.send({ to: ['+258…'], message, from: AFRICASTALKING_SHORTCODE })`; local numbers are normalised to +258. In `transport-client`, SMS copies of USSD results go through `waitUntil()` from `@vercel/functions`, so the USSD reply returns at once while the SMS call finishes in the background of the same invocation.
- Root `lib/africastalking.ts` is a placeholder with the SDK code commented out; the SDK is really used in `lib/notifications.ts`.

4.9 Authentication building blocks

bcryptjs (cost 10) hashes `Utente.senha` and `Administrador.senha`. There are no sessions, JWTs or cookies: login endpoints return the user record, the browser stores it in `localStorage`, and later calls pass ids explicitly. `Motorista.password` is plain text. `NEXTAUTH_SECRET` and `NEXTAUTH_URL` are set in the admin `.env`, but `next-auth` is not installed.

4.10 Testing and tooling

Only `transport-admin` has tests: three Jest + ts-jest suites (`auth`, `crud`, `database`) that run read-only integration checks against the real `DATABASE_URL` (connectivity, counts, relations, bcrypt). Nothing tests the passenger app, the USSD engine, the simulator or the driver portal, and the repository has no CI configuration. ESLint 9 flat config with `eslint-config-next` is set up in every app.

5 Application Architecture

The four deployable units, what each one contains, and how they connect.

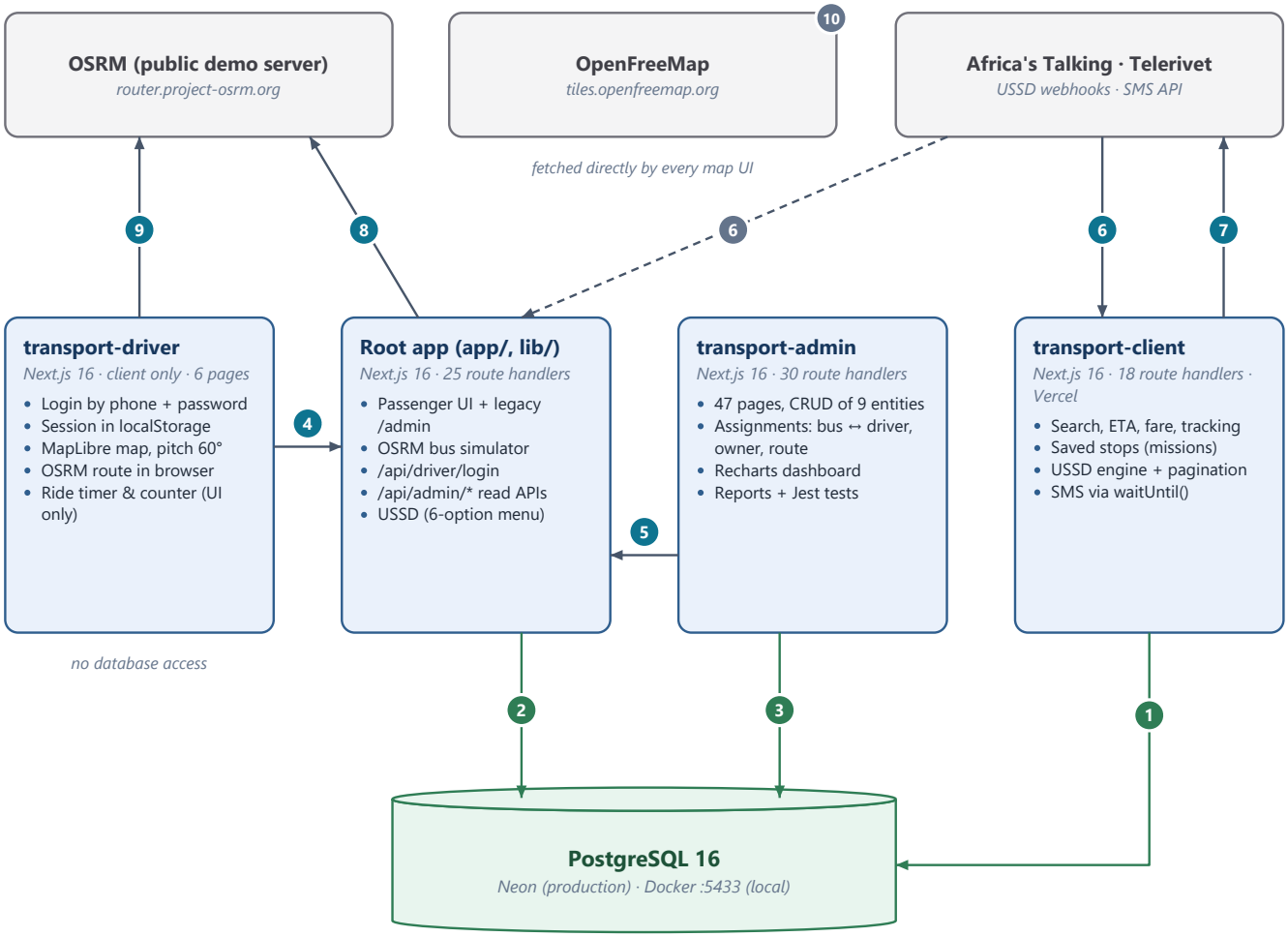


Figure 2 — Containers and their interactions. Numbers refer to the table below; grey badges mark an alternative or indirect path.

#	From → To	Protocol / endpoint	Notes
1–3	client, root, admin → PostgreSQL	Prisma over TLS	Same `DATABASE_URL`; three different schema files
4	transport-driver → root app	`POST /api/driver/login`	`NEXT_PUBLIC_API_URL`, default `http://127.0.0.1:3000`
5	transport-admin → root app	`GET /api/admin/stats`, `/api/buses`, `/api/locations/*`, `/api/admin/*`	Reports page; `NEXT_PUBLIC_API_URL`, default `http://localhost:3002`
6	Gateway → client (or root)	`POST /api/ussd`, `POST /api/ussd-telerivet`	Both apps expose the webhooks; the gateway dashboard decides which one is called
7	transport-client → Africa's Talking	SMS API (HTTPS)	`waitUntil(sendSMS(…))` after USSD replies
8	root app → OSRM	`/route/v1/driving`	Simulator initialisation
9	driver browser → OSRM	`/route/v1/driving` with `steps=true`	Route page
10	all browsers → OpenFreeMap	Style JSON and vector tiles	No API key

5.1 transport-client — passenger app (production)

Page	Purpose
`/`	Full-screen MapLibre map with all buses; calls `/api/startup` to start the simulator
`/auth`	Phone + password login and registration; stores `localStorage['utente']`
`/subscribe`	Subscription form, creates a `Utente` (`mISSION = USER-<timestamp>`, `subscrito = true`)
`/search`	Município → via → paragem (→ destination) selectors; bus list with ETA, trip time and fare
`/track/[id]`	Single-bus view; after the first fetch the marker is animated toward the stop in the browser every 5 s
`/my-transports`	Saved missions
`/user-info`	Profile, password change, missions and buses

- **Libraries.** `lib/prisma.ts`; `lib/busSimulator.ts` (simplified: straight segments, no OSRM, 30 s default); `lib/notifications.ts`; `lib/africastalking.ts`.
- **USSD engine.** `app/api/ussd/route.ts` (1 568 lines) holds the main menu, a context-aware pagination engine, a driver sub-menu and SMS receipts (§ 7.4).

5.2 Root app — parallel passenger app and backend hub

- Has everything the driver portal and the admin reports need: `/api/driver/login`, `/api/driver/health`, `/api/motoristas/[id]` and the read-only `/api/admin/{stats, utentes, motoristas, municipios, provincias, proprietarios}`.
- Holds the **OSRM-based simulator**, `lib/busLocationService.ts` (shared by web and USSD: current position, progress along the route, and a hard-coded table of street names per route code such as `VIA-1A` and `VIA-MAT-BAI`) and `lib/neighborhoodService.ts` (infers region Maputo / Matola and neighbourhood from coordinates and stop names).
- Carries a legacy admin UI under `app/admin/*` (buses, routes, stops, municipalities, users, reports).
- Its USSD (2 175 lines) uses an older six-option menu (help = 5, driver area = 6).

5.3 transport-admin — back-office

- **Modules:** administradores, províncias, municípios, vias, paragens, transportes, motoristas (+ atribuir), proprietários, utentes, relatórios (+ personalizado, agendar, histórico, preview), dashboard, configurações, perfil, login.
- **Per-module pattern:** list page → `[id]` detail → `[id]/editar` → `novo` / `criar`, backed by `/api/<entity>` (paginated, filtered `GET`; `POST`) and `/api/<entity>/[id]` (`GET`, `PUT`/`PATCH`, `DELETE`), plus assignment endpoints for bus ↔ driver and bus ↔ owner.
- **Cross-cutting:** `AdminLayout` (Sidebar + Header), `AuthContext` (localStorage gate and redirect), `ConfirmDialog`, `ResponsiveTable`, `lib/logger.ts` (level from `NEXT_PUBLIC_LOG_LEVEL`).
- **Coupling:** the reports page reads root-app endpoints through `NEXT_PUBLIC_API_URL`.
- **Stray binaries in the source tree:** four vendor PDFs (TruRoute USSD / SMPP specs) and a 3ds Max model (`bus1.max`) inside `app/components/`.

5.4 transport-driver — driver portal

- **Pages:** `/` (login), `/dashboard` (60° tilted map, route line, trip timer, passenger counter, speedometer fed by the browser Geolocation API), `/route` (OSRM geometry through the assigned stops), `/stats`, `/report`, `/support`.
- **Data:** only the login response kept in `localStorage['motorista']`. Nothing is written back: GPS positions, trips, reports and support requests stay in the browser.

6 Data Architecture

Fourteen Prisma models in five domains, stored in one PostgreSQL database.

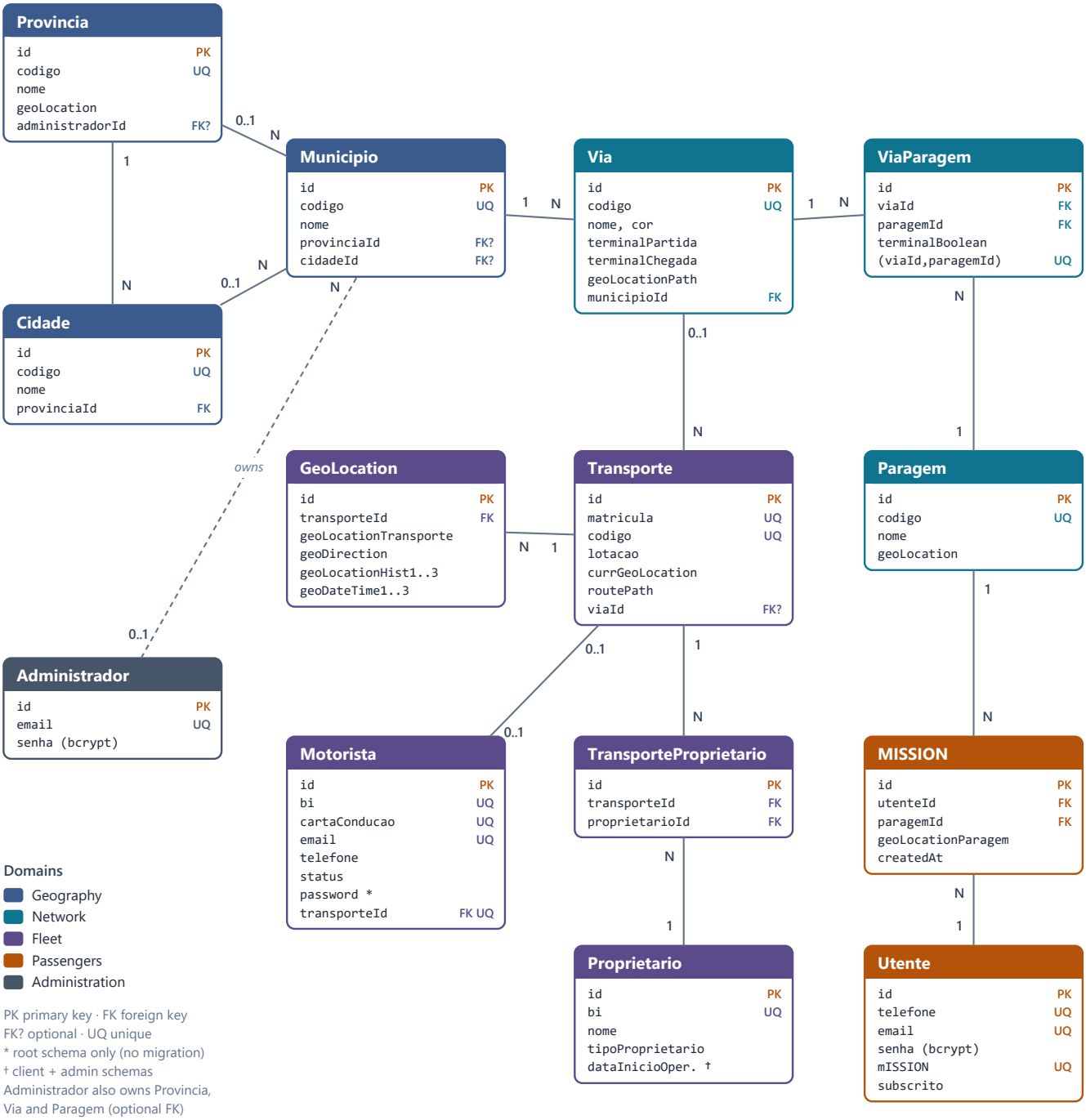


Figure 3 — Entity-relationship model (key fields only), from `prisma/schema.prisma`.

6.1 Entity catalogue

Entity	Meaning	Notable rules
Provincia · Cidade · Municipio	Administrative geography	Unique `codigo`; Municipio links to Provincia and Cidade (both optional)
Via	Bus route / line	Belongs to one Municipio; `cor` sets the map colour; `geoLocationPath` is the route polyline
Paragem	Bus stop	`geoLocation` is "lat,lng"
ViaParagem	Stops on a route	Unique (viaId, paragemId); `terminalBoolean` marks terminals; stop order relies on `id` ordering (no sequence column)
Transporte	Vehicle	Unique `matricula` and numeric `codigo`; optional `viaId`; optional per-bus `routePath`; `currGeoLocation` written by the simulator
GeoLocation	Latest position and short history	One row per bus, updated in place: current point, direction (Indo / Voltando) and three history points
Motorista	Driver	Unique BI, licence and e-mail; `transporteId @unique` gives one driver per bus; `status` ativo / ONLINE / OFFLINE
Proprietario · TransporteProprietario	Owners (companies) and ownership links	Many-to-many with cascade delete
Utente	Passenger	Unique phone and e-mail; bcrypt `senha`; `mISSION` unique string
MISSION	A stop saved by a passenger	Drives arrival SMS for 24 h after creation
Administrador	Back-office user	bcrypt `senha`; optional owner of geography, routes and stops

6.2 Encoding conventions

```
Paragem.geoLocation          "-25.9734,32.5694"                lat,lng
Transporte.currGeoLocation   "-25.9692,32.5732"                lat,lng
Via.geoLocationPath          "32.5694,-25.9734;32.5720,-25.9700" lng,lat;lng,lat;...  (GeoJSON order)
Transporte.routePath         same format as geoLocationPath, overrides the via path per bus
GeoLocation.geoDirection     "Indo" | "Voltando"
Utente.mISSION               "USER-1715430000000"
```

Points use lat,lng and paths use lng,lat. Every parser has to know which is which, and a swap silently puts a bus in the ocean. A PostGIS `geography(Point)` / `LineString` column would remove the ambiguity and allow indexed proximity queries.

6.3 Schema drift

Field	root `prisma/`	`transport-client/prisma`	`transport-admin/prisma`
`Municipio.geoLocation`	—	—	`String?`
`Proprietario.birthDate`	`DateTime`	—	—
`Proprietario.dataInicioOperacoes`	—	`DateTime`	`DateTime`
`Proprietario.foto`, `certificado`	—	`String?`	`String?`
`Motorista.password`	`String?` (no migration)	—	—
Migrations	3 (init → motorista fields)	2	4 (rename birthDate, optional via, município geo)

- If the admin's `rename_birthdate_to_datainicio` migration is applied, root `GET /api/admin/proprietarios` (which selects `birthDate`) fails.
- Drivers created in the admin get `password = NULL`, which the root web login treats as "any password is valid".
- `transport-client` cannot read `Motorista.password` at all, so its USSD driver login only accepts `123456`, the same for every driver.

7 Runtime Behaviour

The main flows, step by step, with the exact calls involved.

7.1 Bus position simulation

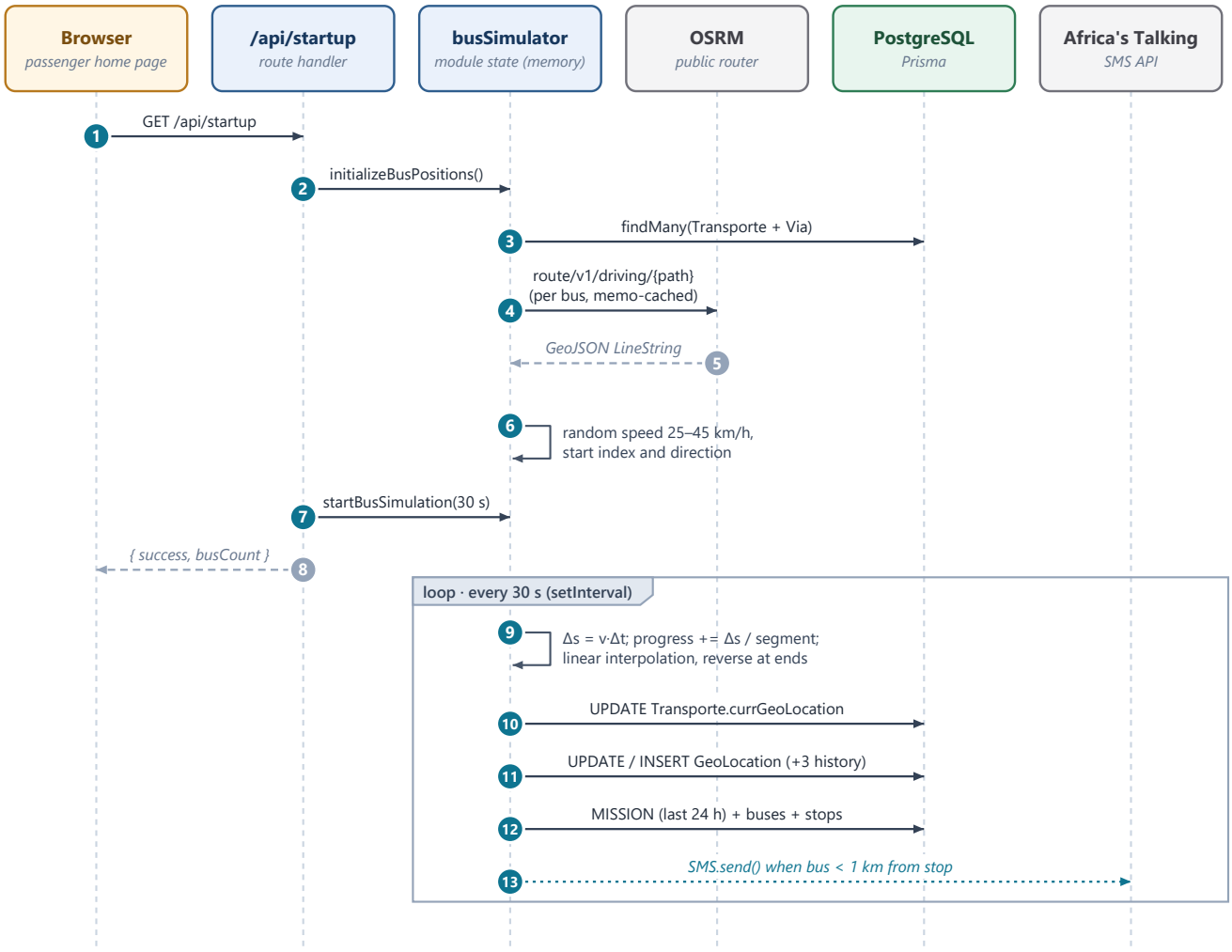


Figure 4 — Simulator start-up and tick (root `lib/busSimulator.ts`).

- **State.** `busPositions: Map<transporteId, {routePath, currentIndex, progress, direction, speed}>` and `routeCache` live in module memory. They are lost on every cold start and are not shared between server instances.
- **Movement.** Each tick advances `progress += v·Δt / segmentLength` along the OSRM polyline, moves to the next vertex when progress reaches 1 and reverses at either end (ping-pong).
- **History.** The three history points are the three previous polyline vertices, and their timestamps are set to now − 5, 10 and 15 min. They are not real observations.
- **Alerts.** Each tick loads every mission of the last 24 h and every bus with its stops, then sends an SMS whenever a bus on a matching via is within 1 km. Nothing records that a passenger was already notified, so the SMS is repeated every 30 s while the bus stays within range.

7.2 Passenger search and ETA

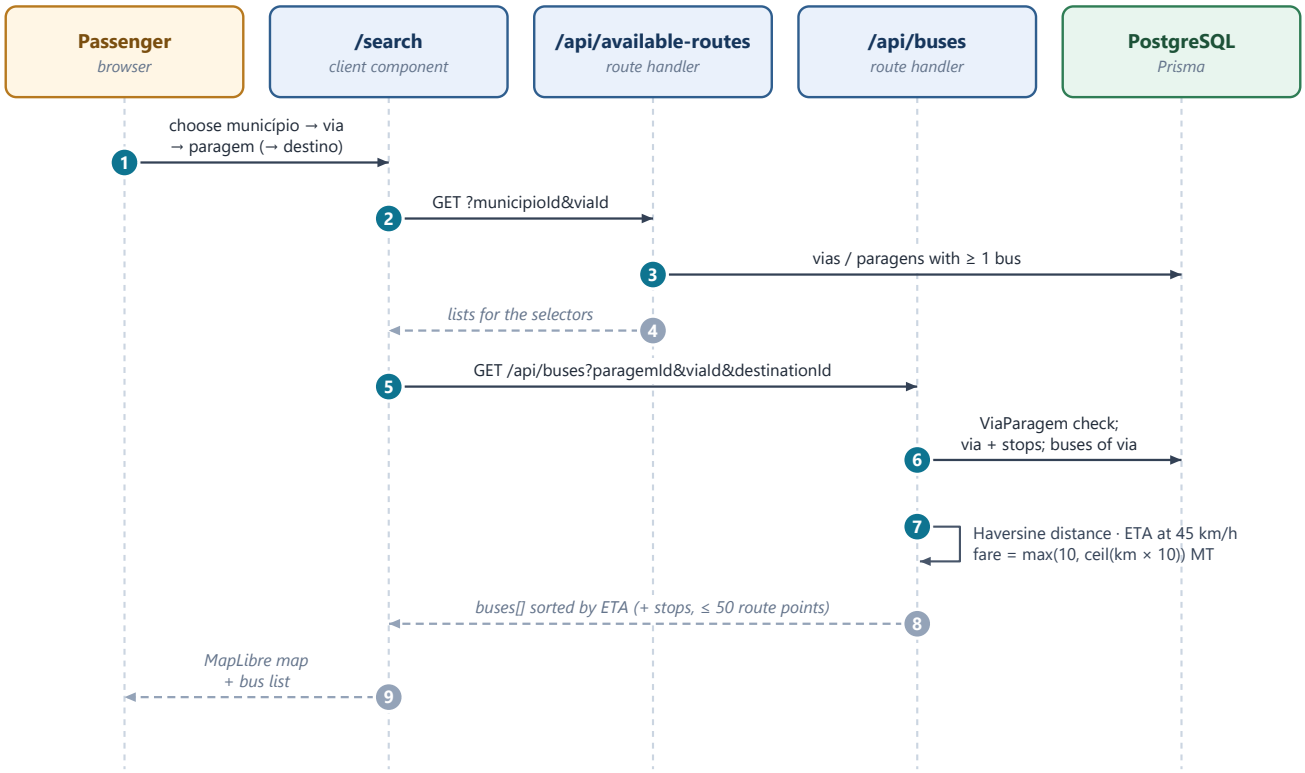


Figure 5 — Search flow in `transport-client`.

`/api/buses` never returns an error: a missing stop, missing via or empty route falls back to "all buses" sorted by distance, and exceptions return `{ buses: [], error }` with HTTP 200. The UI always has something to draw, but monitoring cannot tell failures from real results.

7.3 Tracking page

`/track/[id]` loads the bus once and then runs a 5-second `setInterval` that moves the marker toward the passenger's stop in the browser, using planar distance (degrees × 111 km). The countdown therefore reflects a local animation, not the simulator's database positions.

7.4 USSD session engine

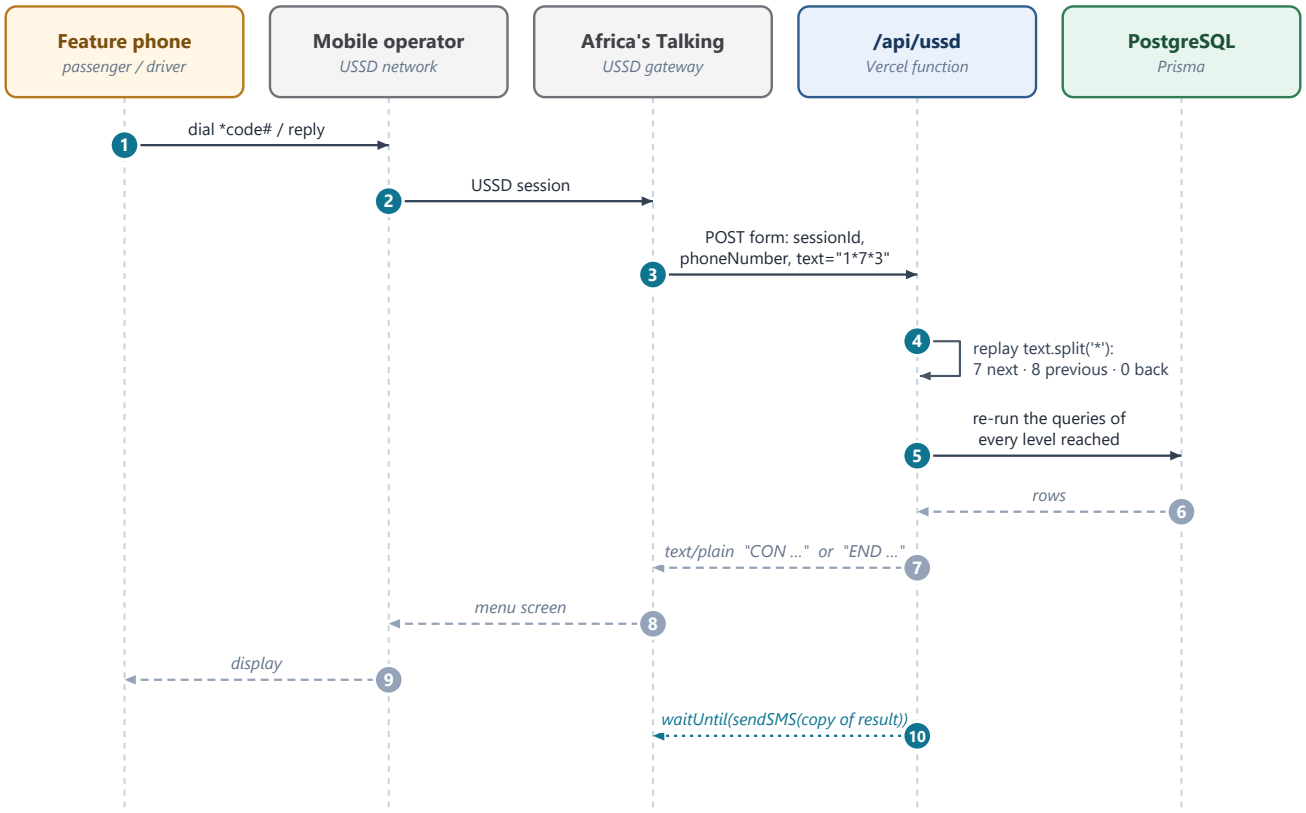


Figure 6 — One USSD hop. The gateway calls the webhook once per user reply.

Stateless replay with pagination

Nothing is stored between hops. On each request the handler rebuilds the menu position from `text`: `7` and `8` move the page of the current list, `0` pops the last choice, and anything else is a selection recorded together with the page it was made on. The chosen item is `list[page × 6 + choice − 1]`.

```
text = "1*7*3"
  '1' -> inputs ['1']        pages [0]      main menu: Encontrar Transporte
  '7' -> page = 1                           next page of the origin list (items 7-12)
  '3' -> inputs ['1','3']    pages [0,1]    origin = locations[1*6 + 3 - 1] = locations[8]
level = 2  ->  show the paged destination list for that origin
```

At the top level (no previous input) `7` is the Ajuda option. Further down, `7`, `8` and `0` are reserved in every list. Because the whole path is replayed, each hop re-runs the queries of every level reached (e.g. `getAvailableLocations()` → `getAvailableDestinations()` → `findTransportInfo()`), so latency grows with menu depth and has to stay below the gateway timeout. The `scratch/time_*query.js` scripts measure exactly this.

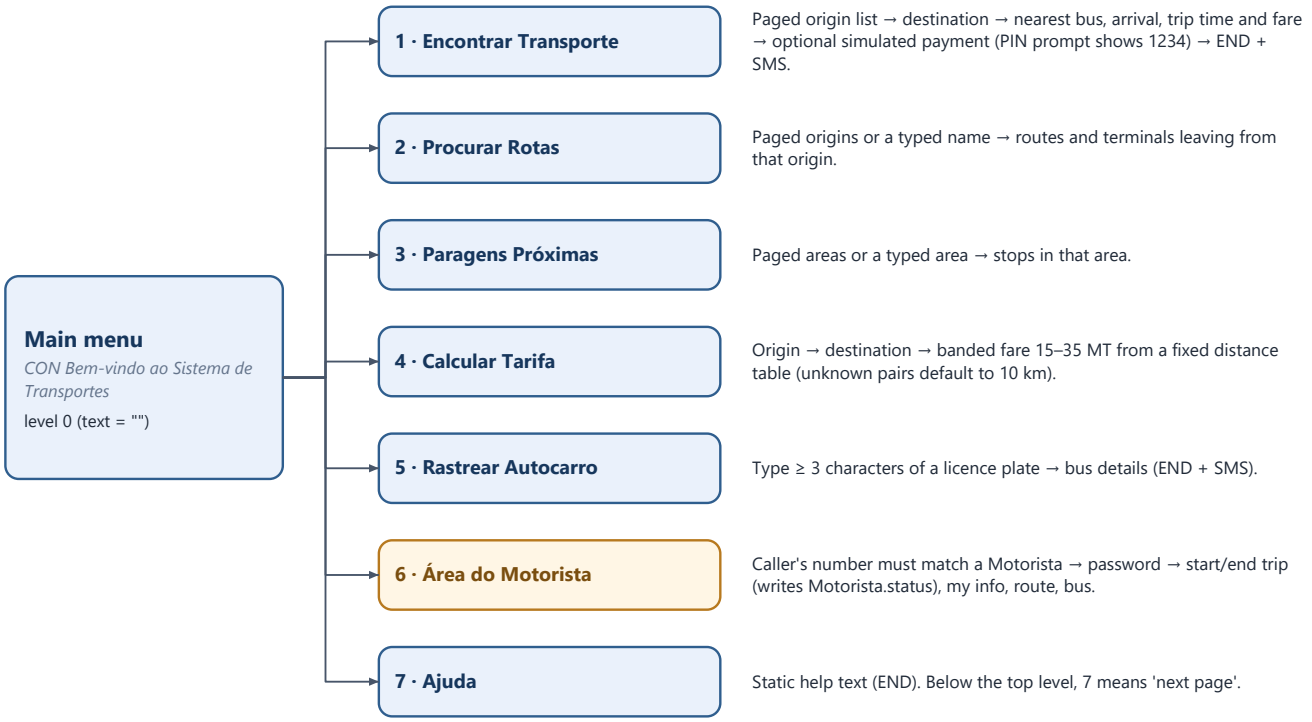


Figure 7 — USSD main menu in `transport-client`.

7.5 Driver portal login

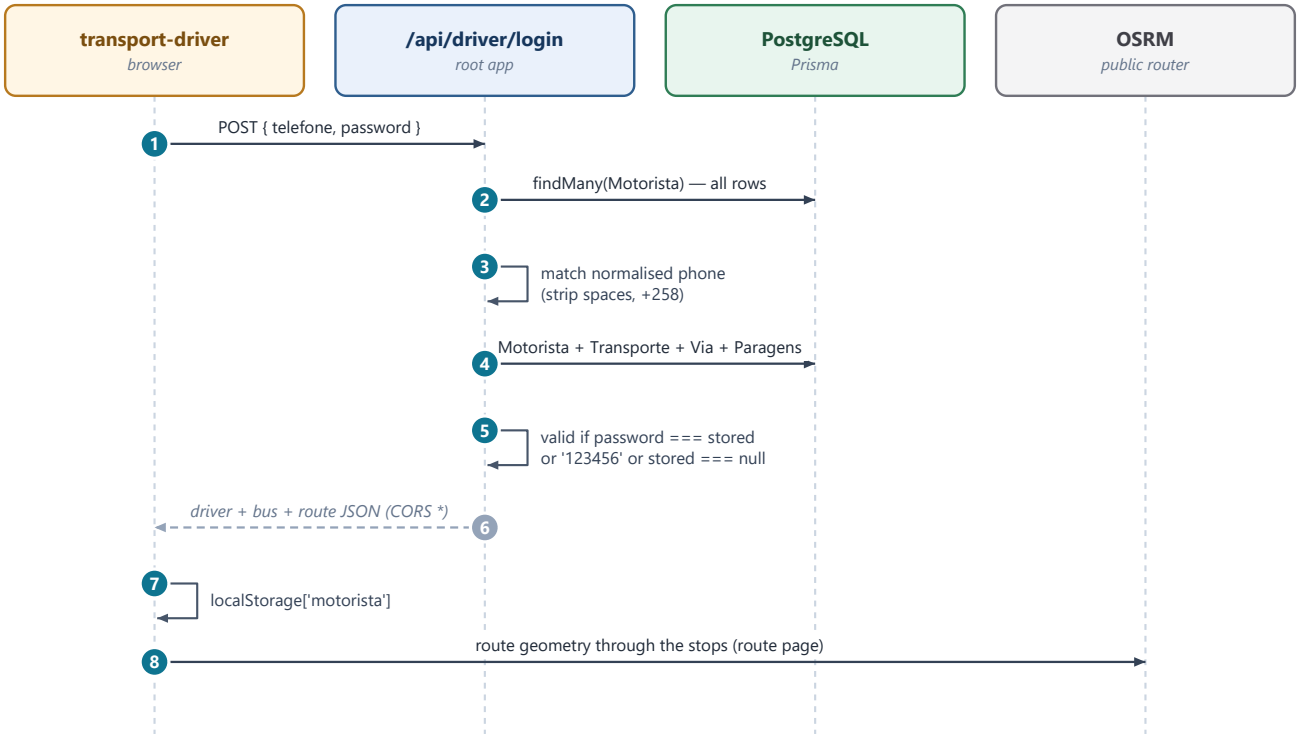


Figure 8 — Driver login (root `app/api/driver/login`).

7.6 Administration

`POST /api/auth/admin/login` checks the bcrypt hash and returns `{ admin }`. The login page stores it in `localStorage['admin']`; `AuthContext` reads it on mount and redirects to `/login` when it is missing. Every CRUD request then goes straight to the admin's own Route Handlers with no credential attached.

8 Domain Calculations

Formulas currently encoded in the code. Several channels compute the same quantity differently.

Quantity	Formula	Where
Great-circle distance	Haversine, R = 6 371 km	`lib/busSimulator.ts`, `/api/buses`, USSD
Bus → stop ETA (web)	ceil(d_km / 45 × 60) min; d < 50 m → 2 min; 0 → 1 min	`/api/buses`
Trip time (web)	straight-line pickup → destination at 45 km/h	`/api/buses`
Fare (web)	max(10, ceil(km × 10)) MT on straight-line km	`/api/buses`
Fare (USSD option 1)	Haversine × 1.3 road factor, then max(10, ceil(km × 10)) MT	`findTransportInfo()`
Fare (USSD option 4)	Fixed pair table (default 10 km) → bands 15 / 20 / 25 / 30 / 35 MT; time at 30 km/h	`calculateFare()`
Simulator step	Δs = v·Δt, v between 25 and 45 km/h; linear lat/lng interpolation	`updateBusPosition()`
Arrival alert	Bus on the mission's via within 1 000 m; ETA at 45 km/h	`checkAndNotifyUsers()`
Route progress	index of the nearest `geoLocationPath` vertex (planar degrees) / (n − 1)	`busLocationService`
Street label	waypoint = floor(p·(n−1)); local < 0.3 "Em …", > 0.7 "Próximo de …", else "Entre … e …"	`getCurrentStreetLocation()`
Tracking animation	planar distance × 111 km, step every 5 s in the browser	`/track/[id]`
USSD list item	list[page × 6 + choice − 1]	`ussd/route.ts`

Inconsistent prices

The same trip can be quoted at three different prices depending on whether the passenger uses the web app, USSD option 1 or USSD option 4. Move fares, speeds and road factors into one shared module (or a `Tarifa` table) used by every channel.

9 Deployment and Operations

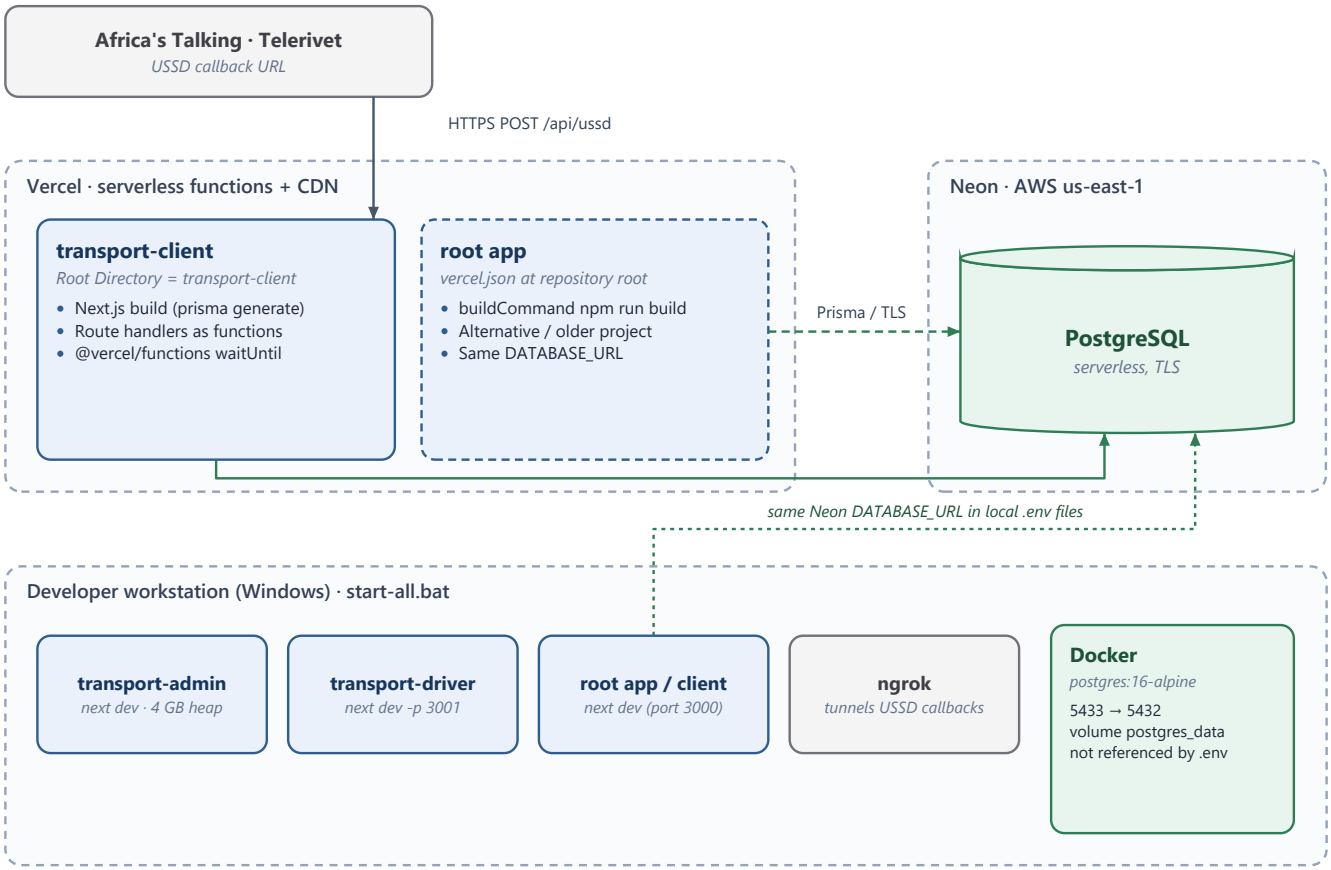


Figure 9 — Deployment view as configured in the repository and local environment files.

9.1 Environments

- **Production:** a Vercel project with Root Directory `transport-client` (see `VERCEL_ROOT_DIRECTORY_FIX.md`). The build runs `prisma generate && next build`, and `postinstall` regenerates the client.
- **Database:** Neon serverless PostgreSQL in AWS us-east-1. The root, client and admin `.env` files hold the same connection string, so local development works against the same database, and the Docker Postgres is effectively unused.
- **Local USSD testing:** `ngrok.exe` (gitignored) exposes a local dev server to the gateway callback.

9.2 Configuration

Variable	Used by	Purpose
`DATABASE_URL`	root, client, admin	PostgreSQL connection (Neon)
`AFRICASTALKING_USERNAME`, `AFRICASTALKING_API_KEY`	root, client	SMS SDK credentials (sandbox by default)
`AFRICASTALKING_SHORTCODE`	root, client	SMS sender id
`TELERIVET_SECRET`	root, client	Webhook secret (only checked when set)
`ADMIN_SECRET`	root, client	Guards `POST /api/admin/seed` (field `authorization` in the JSON body)
`NEXT_PUBLIC_API_URL`	driver, admin	Base URL of the root app's API
`NEXT_PUBLIC_LOG_LEVEL`	admin	Logger verbosity
`NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`	client (`.env.local`)	Unused
`NEXTAUTH_SECRET`, `NEXTAUTH_URL`	admin `.env`	Unused (`next-auth` not installed)
`NODE_OPTIONS`	root `.env`, admin scripts	Larger heap for dev / build

9.3 Local ports

App	`start-all.bat` says	`npm run dev` actually runs	Consequence
transport-admin	3001	`next dev` (3000 by default)	Takes 3000 if started first
transport-client	3000	`next dev` (3000 by default)	Moves to the next free port
root app	—	`next dev -p 3000`	Fails if 3000 is busy
transport-driver	3002	`next dev -p 3001`	Expects the API at `NEXT_PUBLIC_API_URL`

9.4 Observability

Logging is `console.log` with emoji prefixes (and the admin's level-filtered `logger`). There is no structured logging, request id, metrics, error tracking or health dashboard; `/api/driver/health`, `/api/test`, `/api/test-db` and `/api/debug-vias` serve as ad-hoc probes and are publicly reachable. USSD request logs include the full `text`, which contains driver passwords typed in option 6.

10 Security Assessment

Findings from reading the code at this snapshot. Severity assumes the apps are reachable from the internet, as the deployed passenger app is.

Severity	Finding	Where	Recommendation
Critical	No server-side authentication or authorisation on admin APIs; the admin "login" only sets `localStorage`.	`transport-admin/app/api/**`	Session cookies (e.g. Auth.js) checked in `proxy.ts` / middleware and in every handler; roles.
Critical	Driver login accepts `123456` for every driver and any password when the stored one is `NULL`; passwords are stored in plain text.	root `/api/driver/login`, both USSD handlers	Remove the master password, hash with bcrypt, force a reset for drivers without a password.
High	Unauthenticated `GET /api/admin/utentes` returns every passenger's e-mail and phone with `Access-Control-Allow-Origin: *`.	root `app/api/admin/*`	Require an admin session, drop the wildcard CORS.
High	Identity is a client-held JSON blob; endpoints trust ids in the URL (`GET`/`PUT /api/user/[id]`, missions).	`transport-client/app/api/user/**`	Derive the user from a signed session, never from the path.
Medium	Driver passwords typed over USSD travel inside `text` and are written to logs.	USSD `console.log` of the request	Mask `text` in logs; use a PIN with lockout.
Medium	Webhooks are not authenticated: the Telerivet secret is optional and Africa's Talking calls are not verified.	`/api/ussd`, `/api/ussd-telerivet`	Make the secret mandatory; allow-list gateway IPs.
Medium	Unauthenticated `POST /api/simulation` starts / stops the simulator; debug endpoints are public.	root and client	Remove or protect behind an admin session.
Medium	USSD payment flow is simulated (PIN prompt displays 1234) but answers "Pagamento confirmado".	client USSD option 1	Label it as a demo or integrate a real wallet (e.g. M-Pesa) with server-side confirmation.
Low	Development DB credentials in `docker-compose.yml`; local `.env` files point at the shared Neon database.	repository / workstation	Separate dev and prod databases; use a local `.env.example`.

11 Quality Attributes and Technical Debt

Area	Observation	Impact	Recommendation
Maintainability	Two diverged passenger apps; three Prisma schemas; street-name tables duplicated in `busLocationService` and USSD; three fare implementations.	Fixes land in one copy only; behaviour differs by channel.	Merge root `app/` into `transport-client`; one shared `packages/db` (schema + client) and `packages/domain` (fares, ETA, geo) in an npm / pnpm workspace.
Reliability	Simulator in `setInterval` inside serverless functions; started by a page visit.	Frozen, jumping or doubled movement; repeated SMS.	Run movement as a scheduled job (Vercel Cron / worker) with state in the DB, or ingest real GPS from the driver app.
Performance	Driver lookup loads every `Motorista`; each tick loads all buses with all stops; USSD replays queries for every level; geo filtering happens in JS.	Linear growth with fleet and menu depth; gateway timeouts.	Indexed lookups by normalised phone; PostGIS with `ST_DWithin`; cache menu lists per session.
Data quality	Fabricated history timestamps; implicit stop order; lat/lng vs lng/lat formats.	Misleading analytics; ordering bugs.	Real timestamps; `ViaParagem.ordem`; typed geometry columns.
Correctness	`motoristaId` filter on Transporte; `/api/buses` hides errors behind HTTP 200 fallbacks.	Runtime errors; silent failures.	Remove `any`, add API contract tests, return proper status codes.
Footprint	Unused three.js, Leaflet, mapbox-gl and Google loader; PDFs and a .max file in `app/`.	Install / build time.	Prune dependencies and move binaries out of the app.
Testing / CI	Only read-only admin integration tests; no CI.	Regressions reach production unseen.	CI running lint, type-check, unit tests for USSD replay and fare logic, and migrations on a disposable DB.

12 Recommended Roadmap

Ordered by risk reduction per unit of effort.

Phase	Goal	Main changes
1 · Secure	Close the critical holes	Server-side sessions and a guard for admin and user APIs; remove the master driver password and hash driver passwords; drop wildcard CORS on data endpoints; mask USSD logs; require webhook secrets.
2 · Consolidate	One source of truth	Merge the root app into `transport-client`; a workspace with shared `db` and `domain` packages; one migration history; delete one-off scripts or move them to `tools/`; prune unused dependencies.
3 · Make movement real	Trustworthy positions	Driver app posts GPS (`watchPosition` → `POST /api/driver/position`); a scheduled job replaces the in-memory simulator for demos; PostGIS geometry columns; de-duplicated arrival alerts.
4 · Operate	Run it with confidence	CI pipeline, structured logging, error tracking, separate dev / staging / prod databases, deployment config for the admin and driver apps.

13 Key Findings and Future Improvements

What stands out in the current codebase, and where it should go next.

13.1 Key findings

Two diverged copies of the passenger app

- The root `app/` + `lib/` + `prisma/` and `transport-client/` started as the same app and have evolved separately (26 and 45 commits since May). Each has endpoints the other lacks, and `lib/busSimulator.ts` differs (only the root one uses OSRM).
- The driver portal and the admin reports depend on root-only endpoints, while the deployed USSD lives in `transport-client`.

Authentication is client-side only

- No API checks a session: admin CRUD endpoints, root `/api/admin/*` (which returns every passenger's email and phone with `Access-Control-Allow-Origin: *`) and user endpoints accept any caller.
- Driver login accepts the universal password `123456`, and any password when the stored one is `NULL`; driver passwords are stored in plain text.

Simulator is incompatible with serverless hosting

- Bus state lives in module memory and advances on `setInterval`. Short-lived, parallel Vercel instances make positions freeze, jump or advance twice; arrival SMS repeat on every tick.

Schema drift between three schema.prisma copies

- Admin migrations renamed `Proprietario.birthDate` to `dataInicioOperacoes` and added fields; the root schema still reads `birthDate`, and only the root schema knows `Motorista.password` (no migration creates it).

13.2 Future improvements

Area	Improvement
One passenger app	Merge the root `app/` into `transport-client`, with shared database and domain packages, one `schema.prisma` and one migration history.
Authentication	Server-side sessions for administrators, passengers and drivers, hashed driver passwords and request guards on every API.
Live GPS	The driver app sends real positions; a scheduled job replaces the in-memory simulator.
Spatial data	PostGIS geometry columns and indexed proximity queries instead of coordinates stored as text.
Fares and ETA	One shared calculation used by the web app and every USSD option.
Arrival alerts	Notify each passenger once per approaching bus, with a configurable distance.
Payments	Real mobile-money integration (e.g. M-Pesa) for the USSD payment step.
Operations	CI pipeline, automated tests for USSD and fares, structured logging, separate development and production databases.

The phased plan for these changes is in § 12, Recommended Roadmap.

A API Catalogue

A.1 transport-client

Endpoint	Methods	Purpose
`/api/auth/login`, `/api/auth/register`	POST	Passenger login (bcrypt) / registration
`/api/available-routes`	GET	Municípios, vias and paragens that have at least one bus
`/api/buses`	GET	All buses, or buses of a via for a stop with ETA and fare
`/api/bus/[id]`	GET	One bus with its route and stops
`/api/locations`	GET	Municípios, vias, paragens for selectors
`/api/user/[id]`	GET, PUT	Profile read / rename
`/api/user/[id]/change-password`	POST	Change password (requires the current one)
`/api/user/missions`, `/save`, `/[id]`	GET, POST, DELETE	List, create, delete saved stops
`/api/startup`, `/api/simulation`	GET, POST	Start and inspect / control the simulator
`/api/ussd`, `/api/ussd-telerivet`	POST	USSD webhooks
`/api/admin/seed`	GET, POST	Counts / seed (POST guarded by `ADMIN_SECRET`)
`/api/test`, `/api/test-db`	GET	Diagnostics

A.2 Root app

Endpoint	Methods	Purpose
`/api/driver/login`	POST, OPTIONS	Driver login, returns bus, via and stops (CORS *)
`/api/driver/health`	GET, OPTIONS	Probe for the driver portal
`/api/motoristas/[id]`	GET, PUT	Driver read / update
`/api/admin/stats`	GET	Entity counts for admin reports
`/api/admin/{utentes, motoristas, municipios, provincias, proprietarios}`	GET (+ POST motoristas)	Read-only lists for admin reports (CORS *)
`/api/locations`, `/locations/vias`, `/locations/paragens`	GET	Selector data
`/api/buses`, `/api/bus/[id]`	GET	Bus positions, ETA, fare
`/api/auth/*`, `/api/user/missions*`	POST, GET	Passenger auth and missions
`/api/startup`, `/api/simulation`	GET, POST	OSRM simulator control
`/api/ussd`, `/api/ussd-telerivet`	POST	USSD webhooks (older menu)
`/api/admin/seed`, `/api/test`, `/api/test-data`	GET, POST	Seed and diagnostics

A.3 transport-admin

Endpoint	Methods	Purpose
`/api/auth/admin/login`	POST	Admin login (bcrypt)
`/api/{provincias, municipios, vias, paragens, proprietarios, utentes, administradores}`	GET, POST	Paginated lists with filters; create
`/api/{…same}/[id]`	GET, PUT / PATCH, DELETE	Read, update, delete one record
`/api/administradores/[id]/change-password`	POST	Change admin password
`/api/transportes`, `/api/transportes/[id]`	GET, POST, PUT, DELETE	Fleet CRUD
`/api/transportes/[id]/atribuir-motorista`	POST, DELETE	Assign / unassign driver
`/api/transportes/[id]/atribuir-proprietario`, `/proprietario`	POST, DELETE	Assign / unassign owner
`/api/motoristas`, `/[id]`, `/atribuir`	GET, POST, PUT, DELETE	Driver CRUD and bulk assignment
`/api/dashboard/stats`	GET	Dashboard figures
`/api/assign-transportes`, `/api/create-transportes`	POST	Bulk maintenance operations
`/api/check-assignments`, `/api/debug-vias`, `/api/test-db`	GET	Diagnostics

B Glossary (Portuguese → English)

Term	Meaning	Term	Meaning
Via	Bus route / line	Paragem	Bus stop
ViaParagem	Stop on a route	Transporte / autocarro	Vehicle / bus
Matrícula	Licence plate	Lotação	Seating capacity
Motorista	Driver	Carta de condução	Driving licence
BI	National identity card	Proprietário	Vehicle owner (company)
Utente	Passenger / registered user	MISSION / missão	Stop saved for arrival alerts
Província · Cidade · Município	Province · city · municipality	Terminal partida / chegada	Origin / destination terminal
Tarifa	Fare	MT	Mozambican metical
Indo / Voltando	Outbound / inbound	Rastrear	Track
Iniciar / Encerrar viagem	Start / end trip	Ajuda	Help

Method: generated from a static reading of the repository at commit `57b723f` (source files, Prisma schemas, migrations, configuration and the variable names in local `.env` files; no secret values were read). The live database and the Vercel project settings were not inspected.